



Cairo University
Faculty of Engineering
Department of Computer Engineering

Accelera: Graph-Based ML Pipelines, Benchmarking Platform, and Multi-Target Model Deployment

Pipeline Parallelism, Web-Based Leaderboards, and Containerized
Deployments



A Graduation Project Individual Report Submitted
to
Faculty of Engineering, Cairo University
in Partial Fulfillment of the Requirements for the Degree
of
Bachelor of Science in Computer Engineering.

Presented by

Mazen Adel
Student ID: 9220625

Supervised by

Dr. Salma Abdel Monem

June 2026

All rights reserved. This report may not be reproduced in whole or in part, by photocopying or other means, without the permission of the author or department.

Abstract

This report presents the individual design, implementation, verification, and learning journey behind my contributions to three important parts of the Accelera framework: the Pipeline Module, the Benchmark Module, and the Deployment Module. My work focused on making Accelera more complete as an engineering system, not only as a collection of machine-learning algorithms. The contributions described in this report cover internal execution behavior, user-facing platform features, authentication-related workflows, dashboard visibility, and the final deployment stage that transforms trained models into usable services.

Within the Pipeline Module, Mazen contributed to the C++ back-end execution engine supporting branching workflows, topological sorting, DAG constraints, and duplicate-node optimization. In particular, the `runParallel` functionality was a shared contribution between Mazen Adel and Mohamed Ashraf, focusing on thread-safe graph execution, dependency-aware branch scheduling, counting semaphores, and controlled memory release. Alongside this shared parallel-execution work, Mazen also implemented a native Merge Node supporting class and probability-based majority voting strategies.

Within the Benchmark Module, Mazen developed a Node.js REST API using Express and MongoDB to handle secure user authentication (JWT and bcrypt), dataset uploads, leaderboard management, and validation of submitted predictions against hidden ground truths. He also built a React frontend consisting of dedicated User and Admin Dashboards for task filtering, leaderboard viewing, and administrator-level user management.

Finally, the Deployment Module forms the main contribution of this individual work. It integrates a production-ready FastAPI web server featuring runtime data validation via Great Expectations, a request tracking and telemetry logging layer, a lightweight model Version Control System (VCS) with SHA-1 hash commits and rollback support, and an automated deployment engine targeting Local Docker, Heroku, and remote AWS EC2 instances over SSH. This module was the most technically demanding part of my work because it required connecting machine-learning artifacts, configuration files, server-side APIs, Docker containers, command-line workflows, and remote deployment environments into one coherent system.

Contents

1	Introduction	4
1.1	Project Context and Goals	4
1.2	Mazen Adel's Contributions	4
1.3	Report Organization	5
2	Literature Survey	6
2.1	Machine Learning Pipeline Abstractions	6
2.2	Automatic Concurrency and GIL Release	6
2.3	Web Serving and Data Quality Layers	6
2.4	Multi-Target Deployment Platforms	6
3	System Design and Architecture	7
3.1	Pipeline Module Architecture	7
3.1.1	Merge Node Role in the Pipeline	7
3.1.2	Run Parallel Role in the Pipeline	8
3.2	Benchmark Module Architecture	8
3.2.1	Database Schema Design	8
3.2.2	Dashboard Views	9
3.2.3	Login and Sign-up Flow	9
3.3	Deployment Module Architecture	10
4	Implementation Details	11
4.1	Pipeline Module Parallel Scheduler	11
4.1.1	Execution Flow and GIL Release Implementation	12
4.1.2	Concurrency Semaphores and GPU Serialization	12
4.1.3	Personal Reflection on Parallel Execution	12
4.2	Merge Node and Voting Strategies	13
4.2.1	Voting Strategy Implementation and Edge Cases	13
4.3	Benchmark REST APIs and Dashboards	13
4.3.1	Secure Backend Authentication Routes	13
4.3.2	Submission Processing and Score Validation Route	14
4.3.3	Frontend Dashboards	14
4.3.4	User Dashboard Design	14
4.3.5	Admin Dashboard Design	15
4.3.6	Login and Sign-up Implementation	15
4.4	Deployment Module serving and Validation	15
4.4.1	Personal Motivation and Deployment Scope	15
4.4.2	Model Serving Subsystem	16
4.4.3	Deployment Artifact Preparation	17

4.4.4	Runtime Schema Validation	18
4.4.5	Telemetry and Request Tracking	19
4.5	Multi-Target Deployment Orchestration	19
4.6	Model Version Control (VCS)	21
4.7	Command-Line Interface and CLI Commands	23
4.8	Personal Deployment Challenges and Decisions	24
5	System Testing and Verification	25
5.1	Pipeline Verification	25
5.2	Benchmark Subsystem Verification	25
5.3	Deployment Module Verification	25
5.3.1	Subsystem Verification Details	25
5.3.2	Deployment Test Results	26
6	Personal Engineering Reflection	28
6.1	Individual Challenges	28
6.1.1	Pipeline Module Challenges	28
6.1.2	Benchmark Module Challenges	28
6.1.3	Deployment Module Challenges	28
6.2	Learned Technologies and Skills	29
6.3	Individual Outcomes	29
7	Conclusions and Future Work	31
7.1	Summary of Contributions	31
7.2	Future Extensions	31

List of Figures

3.1	Optimization of Duplicate Node Sharing in DAGs	7
3.2	Access Control matrix for dashboards	9
3.3	Model Serving and Validation Architecture	10
4.1	Overall Deployment Module workflow from model artifacts to serving targets	16
4.2	Deployment Orchestration Workflow	20
4.3	Deployment VCS registry structure	21
4.4	VCS Commit Workflow	22
4.5	VCS Deploy Workflow	22

List of Tables

5.1 Deployment Module Test Suite Verification Summary	27
---	----

Chapter 1: Introduction

1.1 Project Context and Goals

Developing and deploying machine learning pipelines in professional settings requires solving challenges that span different parts of the software lifecycle. A simple, linear script is often sufficient for training a single model on a local workstation. However, a production-grade machine learning system must evaluate multiple models, preprocessing methods, and hyperparameters, run experiments fairly, track version histories, and host models in scalable environments.

My individual work was motivated by this gap between a working experiment and a complete usable system. I wanted my contribution to help Accelera feel less like a narrow model-building prototype and more like a practical platform. This meant working on features that improve how workflows execute internally, how benchmark information is presented to users, how access is separated between normal users and administrators, and how trained models can eventually be packaged and served outside the training environment.

The work therefore covered both backend-oriented and user-facing responsibilities. The Pipeline work required thinking about graph execution, dependency order, merging outputs, and parallel scheduling. The Benchmark work required thinking about dashboards, user journeys, login and signup flows, and role-based interfaces. The Deployment work required learning how models, files, servers, containers, APIs, and commands interact. Together, these contributions helped move Accelera toward a more complete end-to-end machine-learning workflow.

Accelera addresses these challenges by organizing the workflow into separate but integrated modules. This individual work focuses on three essential layers:

1. **The Execution Layer (Pipeline Module):** Runs multi-branch pipelines efficiently on multi-core machines, handling dependencies, branching, merging, and memory management.
2. **The Collaboration Layer (Benchmark Module):** Provides a central, secure platform where different developers can test their models on the same datasets under the same evaluation metrics.
3. **The Serving Layer (Deployment Module):** Acts as the main focus of this work, responsible for packaging trained models, validating input data, saving prediction histories, and pushing services to container platforms.

1.2 Mazen Adel's Contributions

As part of the Accelera development team, the author designed, implemented, and verified the following components:

- **Pipeline Module:** Developed the native C++ `MergeNode` class and voting algorithms. Contributed with Mohamed Ashraf to the concurrent graph runner (`runParallel`),

including dependency-aware scheduling, semaphores, GIL management, and memory optimizations.

- **Benchmark Module:** Programmed the Node.js Express backend and MongoDB schemas for users, benchmarks, and submissions. Designed the React dashboards for users and administrators.
- **Deployment Module:** Designed and built the complete model serving system, which includes the FastAPI server, Uvicorn server, Great Expectations validation layer, telemetry logger, and CLI deployment tools for Docker, Heroku, and AWS EC2. Created the custom Model Version Control (VCS) subsystem.

These contributions were connected by one common goal: improving the practical value of Accelera. The Merge Node and run-parallel work improved the execution layer by making graph workflows more flexible and more capable. The dashboards and authentication pages improved the platform layer by making benchmark interaction clearer for users and administrators. The Deployment module improved the delivery layer by making it possible to move from trained models and saved artifacts toward a real API service. This report discusses these areas as technical work, but also as an individual engineering journey in which each module required a different way of thinking.

1.3 Report Organization

The rest of this report is organized as follows: Chapter 2 reviews the relevant background and related technologies. Chapter 3 describes the architecture, schemas, and design constraints of the three modules. Chapter 4 explains the details of the implementation and algorithms. Chapter 5 presents the testing results and verification, and Chapter 7 concludes with future improvements.

Chapter 2: Literature Survey

2.1 Machine Learning Pipeline Abstractions

Machine learning pipelines are commonly used to organize workflows. The most popular pipeline system is the linear model in `scikit-learn`, which chains transformers and estimators together. However, a linear pipeline cannot naturally represent complex, non-linear workflows. For example, if a developer wants to compare three scaling methods and two classifiers, a linear system requires building six separate pipelines. This duplicates both code and execution time, as early steps must run repeatedly. Modern systems address this by representing workflows as Directed Acyclic Graphs (DAGs), where nodes represent computational steps and edges represent data dependencies.

2.2 Automatic Concurrency and GIL Release

In Python-based frameworks, running pipelines concurrently is limited by the Global Interpreter Lock (GIL). The GIL prevents multiple threads from executing Python bytecodes at the same time. To achieve true parallel execution on multi-core systems, heavy computations must be offloaded to compiled native languages like C++ using binding tools like `pybind11`. When executing C++ code, the Python GIL can be released (`py::gil_scoped_release`), allowing separate native threads to run independently on different CPU cores.

2.3 Web Serving and Data Quality Layers

Model serving requires turning saved binaries (such as pickle files) into active web endpoints. FastAPI is a common choice for this, utilizing asynchronous loops to handle concurrent requests. However, raw inputs submitted by users can contain invalid formats or values that cause models to crash. Data validation tools like Great Expectations help prevent this. They define schema rules (such as expected columns, types, and range boundaries) to check data quality at runtime before sending inputs to the model.

2.4 Multi-Target Deployment Platforms

Deploying serving containers involves different target architectures:

- **Local Docker:** Provides local isolation, ensuring the service runs identically in development and production.
- **Heroku Container Registry:** Simplifies cloud hosting by managing HTTP routing and container scaling automatically.
- **AWS EC2:** Offers raw virtual machines in the cloud, requiring custom SSH management, codebase synchronization, remote container building, and manual firewall configuration.

Chapter 3: System Design and Architecture

3.1 Pipeline Module Architecture

The Pipeline Module represents machine learning workflows as a DAG. Nodes are split into six functional categories: input, preprocessing, model, prediction, metric, and merge. The DAG is compiled by sorting the nodes topologically, which ensures that parent nodes execute before their child nodes.

This module was important to my work because it required understanding a machine-learning workflow as a connected structure rather than a sequential script. In a simple script, the programmer writes the order directly. In a graph system, the engine must infer a valid order from node connections, preserve dependencies, and make sure that data produced by one node is delivered correctly to the next node. Working on this part helped me understand that pipeline design is not only about adding model or preprocessing functions; it is also about controlling how information moves through the system.

Two major optimizations were introduced to improve execution performance. The duplicate-node sharing work is presented as part of Mazen’s pipeline contribution, while the `runParallel` execution functionality was developed as a shared contribution by Mazen Adel and Mohamed Ashraf:

1. **Duplicate first-node sharing:** When multiple branches start with the same preprocessing step, the engine automatically shares the output of the first node rather than duplicating the calculation (Figures 3.1a and 3.1b).
2. **Active consumer counting:** The engine counts the active downstream consumers for each intermediate array. Once all child nodes finish execution, the parent node’s temporary data is freed from RAM.

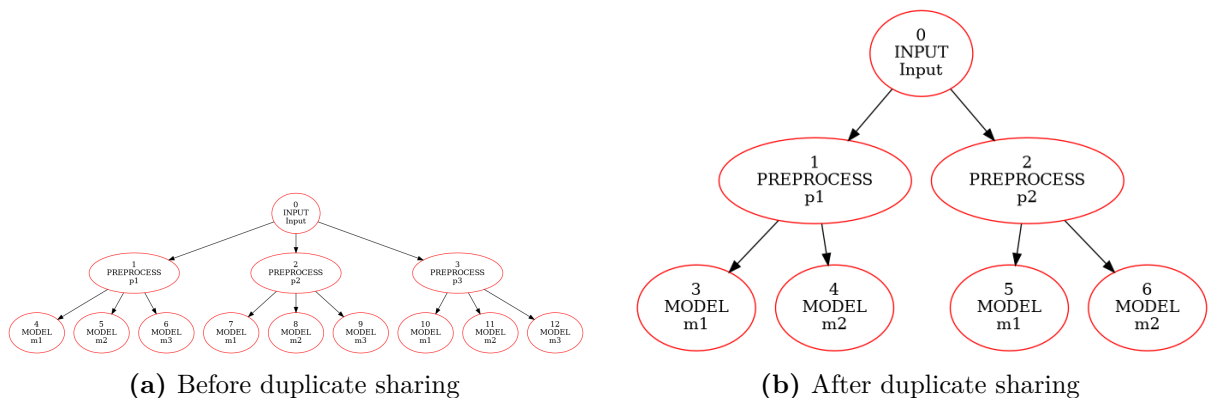


Figure 3.1: Optimization of Duplicate Node Sharing in DAGs

3.1.1 Merge Node Role in the Pipeline

The Merge Node is one of the features that makes the graph pipeline more flexible than a simple linear pipeline. In a branch-based workflow, several paths may produce different

predictions or probability outputs. Without a merge operation, the user would need to collect these outputs manually and write separate code to combine them. The Merge Node gives the graph a controlled point where multiple outputs can be joined into one final result.

From my perspective, implementing the Merge Node was challenging because it required understanding how upstream nodes communicate with downstream nodes. The output shape may differ depending on whether the branch produces direct class labels or probability vectors. Therefore, the merge behavior had to handle both direct predictions and probability-based predictions in a way that remained deterministic and understandable. This work taught me that graph nodes are not isolated functions; each node must respect the data contract between previous and next stages.

3.1.2 Run Parallel Role in the Pipeline

The `runParallel` functionality allows independent graph branches to execute concurrently when their dependencies are satisfied. This improves the execution capability of Accelera because branch-heavy workflows often contain paths that do not depend on each other after a shared preprocessing stage. Running those branches sequentially wastes available CPU resources, while running them in parallel can reduce waiting time and improve the user experience for larger experiments.

The main difficulty in this work was thinking beyond sequential execution. A graph cannot simply start all nodes at once. Each node must wait until its parent nodes have finished, and shared intermediate data must remain available until all consumers are done. For this reason, the parallel scheduler needed synchronization, dependency checking, active consumer counting, and careful GIL handling. This was one of the parts of the project that made me think like a systems engineer rather than only as a machine-learning user.

3.2 Benchmark Module Architecture

The Benchmark Module provides a web-based platform for comparing model performances. It consists of a React frontend and a Node.js backend using Express and MongoDB.

The Benchmark Module was important because it changed Accelera from a local technical framework into a more visible platform. Machine-learning results are only useful when users can inspect them, compare them, and understand where their work stands relative to other submissions. My work on the dashboards and authentication-related screens focused on this product side of the project: making benchmark information accessible and separating the experience of normal users from administrative users.

3.2.1 Database Schema Design

The database schemas are defined using Mongoose. The three main schemas are:

- **User Schema:** Stores names, emails, hashed passwords (via bcrypt), and role properties ("user" or "admin").
- **Benchmark Schema:** Defines the title, description, target column name, problem type, Google Drive dataset links, and a reference to the evaluation metric schema.
- **Submission Schema:** Tracks the submitted repository link, predictions file link,

submission date, and calculated score.

3.2.2 Dashboard Views

The User Dashboard lists active benchmarks and submissions. The Admin Dashboard is restricted to administrative users (Figure 3.2), allowing them to browse and delete users, manage roles, and register new evaluation metrics.

The user dashboard was designed to answer a simple question: what has this user created or submitted? It displays the user's benchmarks and submissions in a clear page so the platform does not feel like a collection of disconnected forms. This matters because benchmark systems depend on visibility. If users cannot easily see their own work, scores, and submitted results, the system becomes difficult to use even if the backend calculations are correct.

The admin dashboard has a different purpose. It is not mainly about personal benchmark progress; it is about system management. The administrator needs to inspect users, control access, and manage platform-level actions. Working on this part helped me understand that interface design depends on role. A normal user needs clarity and personal progress. An administrator needs control, monitoring, and permission-aware actions.

Role	Dashboard Access	Authorized Operations
user	User Dashboard	View leaderboards, create benchmarks, submit predictions
admin	Admin Dashboard	View users, delete users, register new metrics

Figure 3.2: Access Control matrix for dashboards

3.2.3 Login and Sign-up Flow

The login and sign-up pages were also part of making the benchmark module feel like a complete platform. A benchmark platform needs user separation because submissions, dashboards, and administrative permissions depend on knowing who is interacting with the system. The sign-up flow creates accounts, stores hashed passwords through the backend, and assigns roles based on the configured administrator email. The login flow verifies credentials, stores the returned user and token in the browser, and allows the frontend to show the correct pages.

From a personal learning perspective, this part taught me that authentication screens are not only visual forms. They connect the frontend, backend routes, validation messages, token storage, navigation, and role-based access. A small mistake in this flow can make the platform confusing or unsafe. Therefore, I had to think about the full user journey: creating an account, logging in, seeing the dashboard, accessing the admin view only when authorized, and logging out cleanly.

3.3 Deployment Module Architecture

The Deployment Module is the primary contribution of this work. It packages trained estimators and preprocessors into a containerized REST service. Figure 3.3 outlines the serving and validation components.

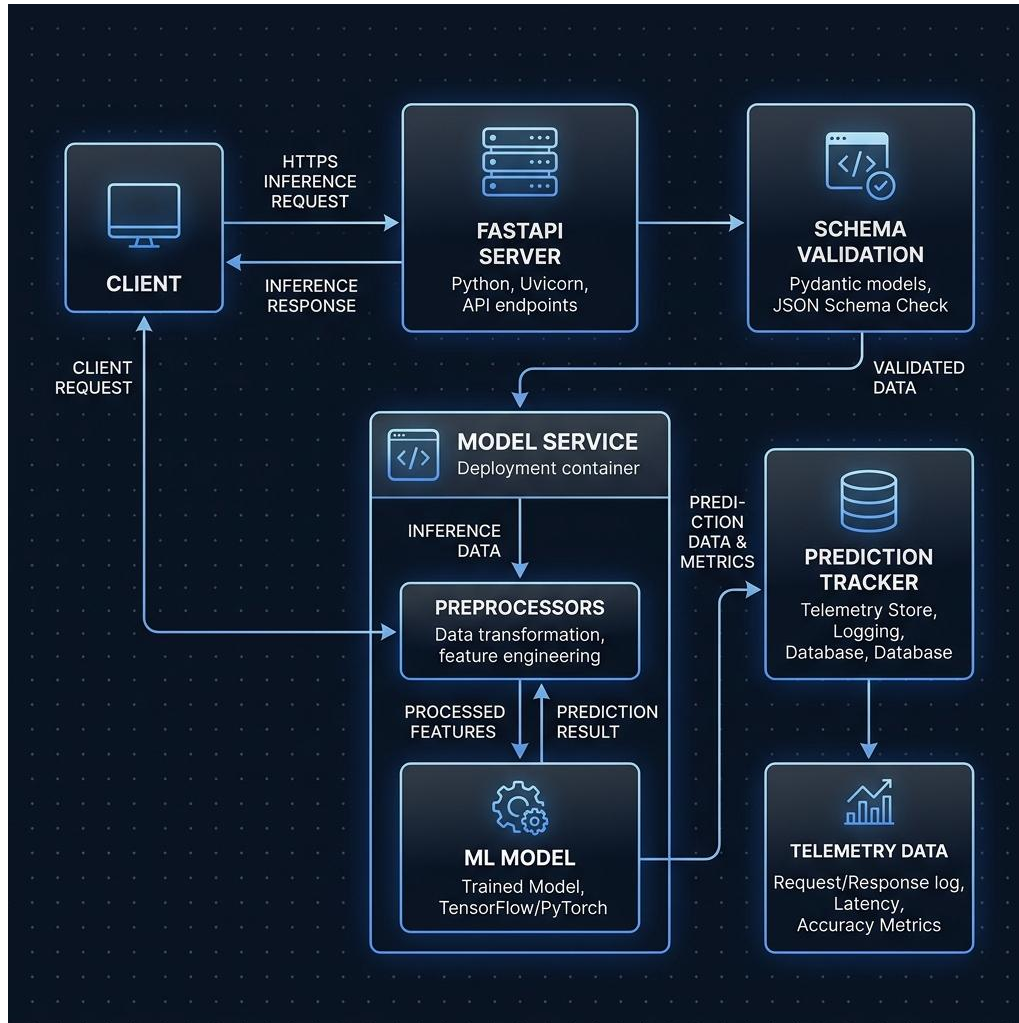


Figure 3.3: Model Serving and Validation Architecture

The system design relies on four core elements:

- **FastAPI Server:** Exposes standard inference endpoints for JSON payloads and batch inference endpoints for uploaded CSV files.
- **Great Expectations Schema Validation:** Validates incoming request values against expected bounds, preventing invalid inputs from reaching the model.
- **Telemetry Tracking:** Logs request inputs, outputs, timestamps, and execution times asynchronously to a JSON lines file on disk.
- **Model Version Control (VCS):** Creates unique commits of configuration and model binaries, allowing rollbacks to older deployments.

Chapter 4: Implementation Details

4.1 Pipeline Module Parallel Scheduler

The parallel scheduler in Accelera is implemented in C++ in `src/core/graph.cpp`. This `runParallel` functionality was a joint contribution by Mazen Adel and Mohamed Ashraf. When parallel execution is enabled, the graph uses `std::async` to execute independent nodes concurrently. The scheduling logic uses counting semaphores to limit the number of active CPU threads, and a binary semaphore to serialize GPU access.

This shared work was important because graph execution cannot be parallelized by simply launching every node at the same time. The scheduler must preserve dependency order, wait until parent nodes finish, protect Python object access through GIL handling, and release intermediate data only after all downstream consumers have finished using it. Mazen’s role in this work focused on understanding how graph branches should be scheduled safely and how memory should be released without breaking later nodes, while Mohamed Ashraf also contributed to the design and implementation of the run-parallel execution flow.

Algorithm 4.1 shows the parallel execution logic. It tracks finished and started nodes, release the GIL during execution, and catches exceptions thrown by separate threads.

Algorithm 4.1: Parallel Execution Engine in C++

```

1: Input: execution_order nodes, cpu_threads limit
2: Output: executed graph with updated node states
3:
4: 1: Release GIL via py::gil_scoped_release
5: 2: Initialize finished_executing map to false for all nodes
6: 3: Initialize started_executing map to false for all nodes
7: 4: Initialize CPU semaphore with cpu_threads count
8: 5: Initialize GPU semaphore with 1 (binary semaphore)
9: 6: Initialize atomic counter rem_nodes = size(execution_order)
10: 7: while rem_nodes > 0 do
11: 8:   Acquire nodes semaphore
12: 9:   for each node in execution_order do
13: 10:    if started_executing[node] == true then continue
14: 11:    check if all parent nodes are finished_executing
15: 12:    if parents_finished == true then
16: 13:      if node.uses_gpu == true then
17: 14:        Acquire GPU semaphore
18: 15:      else
19: 16:        Acquire CPU semaphore
20: 17:      end if
21: 18:      set started_executing[node] = true
22: 19:      spawn std::async(launch::async, [node]() {
23: 20:        Acquire GIL via py::gil_scoped_acquire
24: 21:        try {
25: 22:          node->execute()
26: 23:        } catch (...) {
27: 24:          store_exception()
28: 25:        }
29: 26:        decrement consumer counts for parent nodes
30: 27:        if parent consumer count == 0 then
31: 28:          release parent temporary memory
32: 29:        end if
33: 30:        set finished_executing[node] = true
34: 31:        decrement rem_nodes atomically
35: 32:        Release CPU/GPU semaphores
36: 33:        Release nodes semaphore
37: 34:      })

```

```

38: 35:         end if
39: 36:     end for
40: 37: end while
41: 38: Re-throw any caught exceptions to the Python runtime

```

4.1.1 Execution Flow and GIL Release Implementation

The primary implementation challenge in the parallel execution flow was GIL (Global Interpreter Lock) contention. Because Python objects (such as scikit-learn estimators) are executed inside separate worker threads, concurrent calls will block each other if they attempt to manipulate Python references simultaneously. As part of the shared `runParallel` contribution, Mazen Adel and Mohamed Ashraf structured a two-tier block wrapper in `src/core/graph.cpp`:

1. Before entering the loop-scheduling phase, the main thread releases the GIL using `py::gil_scoped_release`. This allows other native OS threads to run without waiting.
2. Within each spawned worker thread, before invoking the node's `execute()` function, the GIL is explicitly acquired using `py::gil_scoped_acquire`. As soon as the call exits, the scope ends, immediately freeing the GIL for other concurrent threads.

4.1.2 Concurrency Semaphores and GPU Serialization

To control the allocation of hardware resources, the shared `runParallel` implementation uses custom semaphores based on C++ synchronization primitives:

- **CPU Thread Limiter:** A counting semaphore initialized to `ACCELERA_NUM_THREADS` (or system cores minus one). Worker threads block on this semaphore before execution, preventing context-switching overhead from over-subscription.
- **GPU Access Lock:** A binary semaphore (mutex) used to serialize accesses to CUDA contexts. Because multiple threads submitting operations to the same GPU concurrently can cause driver crashes, any node with the `uses_gpu` flag is forced to acquire this binary lock, serializing GPU execution while keeping CPU-bound preprocessing branches running in parallel.

4.1.3 Personal Reflection on Parallel Execution

The most valuable lesson from the `runParallel` work was that performance improvements must not break correctness. It is easy to describe parallel execution as “running things at the same time,” but the real engineering problem is deciding what is allowed to run at the same time. A node can only start when every parent has produced valid output. A parent buffer can only be released when every child that needs it has finished. If either rule is broken, the graph may produce wrong results, crash, or lose intermediate data.

This work helped me understand synchronization as a design responsibility. Semaphores, atomic counters, and maps for started and finished nodes were not only low-level programming tools; they were the mechanism that protected the graph's logical meaning. The experience also showed me why debugging parallel systems is difficult. A sequential bug often appears in the same place every time, while a parallel bug may depend on timing, scheduling, and the order in which threads finish. This made careful design and clear testing essential.

4.2 Merge Node and Voting Strategies

The `MergeNode` class combines outputs from multiple prediction branches. Mazen implemented this node in C++ (`src/nodes/merge.cpp`) and connected it to NumPy and SciPy components. The node supports a hard-voting strategy implemented in C++:

- **Direct Class Predictions (1D Arrays):** Collects predicted category labels from all upstream branches, stacks them into a single 2D NumPy array (rows correspond to samples, columns to model outputs), and uses `scipy.stats.mode` to extract the majority class label.
- **Probability Predictions (3D Arrays):** Stacks predicted probabilities across all branches. It computes class labels for each model using `np.argmax` across the probability dimensions and then applies majority voting on the resulting class labels.

4.2.1 Voting Strategy Implementation and Edge Cases

In `src/nodes/merge.cpp`, Mazen handled several implementation edge cases:

1. **Single-Model Fallback:** If the merge node only receives a single input branch, it directly routes the input to the output without calling voting functions, avoiding unnecessary computation.
2. **Tie-Breaking:** When the voting results are split evenly, the node falls back to selecting the output of the first model (the primary branch), guaranteeing deterministic results.
3. **GIL Safety in Numpy Stack:** Because stack operations call back into the Python/NumPy interpreter, the merge execution block is wrapped in a GIL acquisition scope, securing memory allocations.

The Merge Node was valuable because it made branch comparison more natural. Instead of forcing the user to leave the graph and manually combine model outputs, the graph itself can represent the final aggregation step. This makes the pipeline more expressive and closer to real experimentation, where different model branches may contribute to one final decision. Personally, this part improved my understanding of output shapes, deterministic behavior, and how a node should remain predictable even when receiving different kinds of upstream data.

4.3 Benchmark REST APIs and Dashboards

Mazen designed and built the full client-server subsystem for the Benchmark module.

4.3.1 Secure Backend Authentication Routes

Implemented in Express and MongoDB in `benchmark/backend/routes/user.js`:

- **Login Route (POST /login):** Sanitizes the input email, locates the user document, and compares the password against the stored bcrypt hash. Upon matching, it generates a signed JSON Web Token (JWT) with a 7-day expiration containing the user's MongoDB ID and role payload.

- **Signup Route (POST /signup):** Hashes new user passwords using a salt work factor of 10. If the email matches the environment parameter `ADMIN_EMAIL`, the server automatically promotes the role to "admin".
- **Deletion Route (DELETE /:id):** Restricted to admin users, allowing the deletion of any user profile except their own to prevent lockout.

4.3.2 Submission Processing and Score Validation Route

Implemented in `benchmark/backend/routes/submissions.js`, the scoring route runs the evaluation pipeline when a user submits a CSV file:

1. Downloader modules download the predictions CSV file and the hidden ground truth CSV from Google Drive.
2. The backend validates the integrity of the files, verifying that the number of rows matches.
3. It executes a Python worker subprocess configured with the target metric schema (e.g. F1-score or Mean Squared Error), calculates the performance score, and saves the submission to MongoDB.

4.3.3 Frontend Dashboards

Built in React (`benchmark/frontend/src/components/`) using state Hooks and `async fetch` requests:

- **User Dashboard (`dashboard.jsx`):** Displays a user's details, lists all benchmarks created by the user, and presents a grid of their submissions with performance scores and date history.
- **Admin Dashboard (`adminDashboard.jsx`):** Renders user lists and provides interactive deletion controls, checking JWT payloads to ensure administrative access.

4.3.4 User Dashboard Design

The user dashboard was implemented to give each user a personal overview of their benchmark activity. It retrieves the authenticated user's benchmarks and submissions, displays account information, and presents the user's results in a structured page. This matters because benchmark platforms can become confusing if users must navigate through many pages to remember what they created or submitted. By grouping personal benchmarks and submissions together, the dashboard makes the platform easier to use and gives the user a clear sense of ownership.

While implementing this screen, I learned that a dashboard is not only a place to display data. It must also handle empty states, loading states, and missing authentication. For example, when a user is not logged in, the dashboard redirects the experience toward login instead of failing silently. When no benchmarks or submissions exist, the page displays a clear empty message. These small interface decisions help make the platform feel more complete and understandable.

4.3.5 Admin Dashboard Design

The admin dashboard was designed for a different type of user. Instead of showing personal submissions, it lists registered users and provides management actions such as deleting users while preventing the administrator from deleting their own account. The backend protects this route using authentication and admin middleware, while the frontend checks the current user's role before rendering the management interface.

This part taught me that user roles affect both interface design and backend logic. A normal user interface should reduce complexity and focus on personal work. An admin interface should expose control and monitoring features while remaining protected. Implementing this distinction helped me understand how a web platform becomes more than a set of pages: it becomes a role-aware system.

4.3.6 Login and Sign-up Implementation

The login and sign-up pages connect the frontend to the backend authentication routes. The sign-up page collects the user's name, email, and password, sends them to the backend, and redirects the user to login after account creation. The backend hashes passwords using `bcryptjs` and assigns the admin role when the email matches the configured administrator email. The login page verifies credentials, receives a JSON Web Token (JWT), stores the user and token in local storage, and allows the frontend to send authenticated requests later.

From a personal perspective, this part was important because it connected technical correctness with user experience. The authentication flow had to be simple enough for users, but it also had to support access control for dashboards. I learned that login and sign-up pages are part of the product identity of a platform. If these screens are unclear or disconnected, the rest of the system feels incomplete even if the benchmark calculations work correctly.

4.4 Deployment Module serving and Validation

Mazen was fully responsible for implementing the Deployment module, which converts trained pipelines into containerized web services.

4.4.1 Personal Motivation and Deployment Scope

The Deployment module was my main contribution because it represents the point where Accelera moves from experimentation toward practical use. In many machine-learning projects, the workflow stops after a model is trained, evaluated, and saved to disk. However, a saved model file is not enough for real usage. It must be loaded by a service, protected by validation, exposed through an API, packaged with dependencies, and started on a machine that users or other applications can reach.

At the beginning of this part, deployment was challenging for me because it required knowledge outside the normal model-training workflow. I had to understand how `.pkl` model files, configuration files, Python services, Docker containers, command-line scripts, SSH connections, and cloud/server machines interact. This changed my view of machine-learning engineering. I learned that deployment is not only writing prediction code; it is the process of making a model usable, repeatable, and reachable in a real environment.

Figure 4.1 summarizes the complete scope of the module. It shows how workspace

inputs are committed into the deployment VCS, how the deployment builder prepares the serving container, how validation and inference operate inside the service, and how the final container can run on local Docker, Heroku, or AWS EC2.

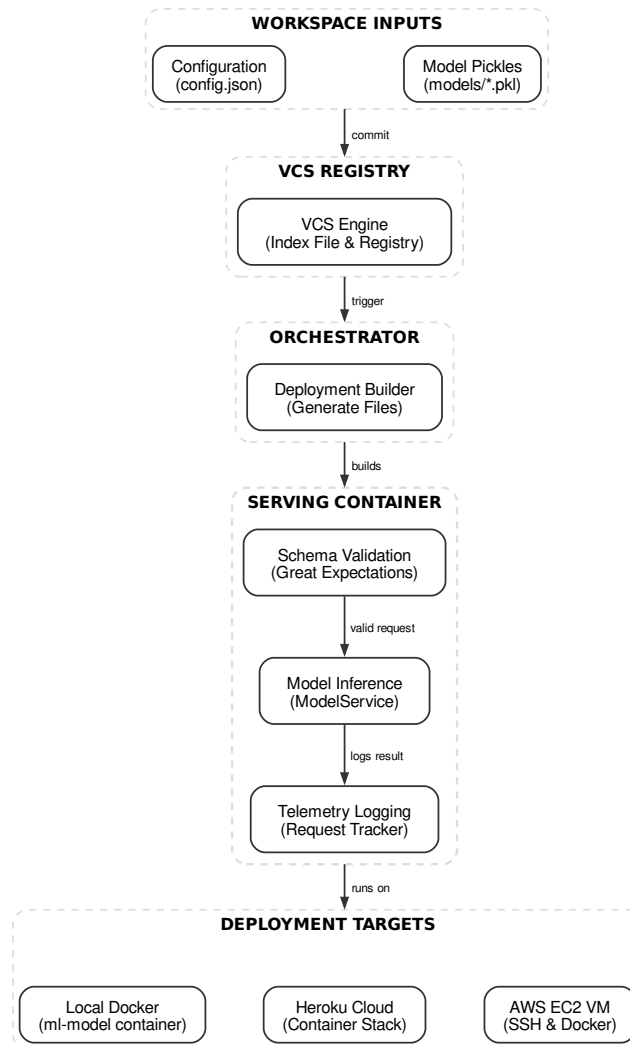


Figure 4.1: Overall Deployment Module workflow from model artifacts to serving targets

This figure was especially useful for organizing the module in my mind. Deployment contains many moving parts, and it is easy to lose track of where one responsibility ends and another begins. Separating workspace inputs, version control, orchestration, serving, and deployment targets helped me design the module as a workflow instead of a random collection of commands.

4.4.2 Model Serving Subsystem

Model serving is built on FastAPI and Uvicorn to handle concurrent HTTP requests. The subsystem loads the model and preprocessors via **ModelService**, which inspects the configuration (**config.json**) to locate the serialized artifacts. During execution, it runs input data through any fitted preprocessors in sequence before invoking the

model’s prediction method. A web-based graphical user interface (GUI) is dynamically rendered based on the expected input features, allowing users to submit single predictions interactively.

The serving subsystem was the first major step in turning model artifacts into a real application. I designed it around a clear separation of responsibilities: `FastAPI` receives requests, `ModelService` loads the model and preprocessing artifacts, `schema_validation.py` protects the model from invalid inputs, and `tracking.py` records prediction activity. This separation made the module easier to test and easier to explain in the documentation.

The following methods and files implement the serving logic:

- `modelservice.py`:
 - `ModelService.load()`: Parses `config.json` to discover the active model configuration. It opens and loads the main estimator file (e.g. `model.pkl`) and any auxiliary preprocessors (e.g. scalers or encoders) from the models directory.
 - `ModelService.predict(input_data, validate=True)`: Performs prediction. If validation is active, it calls `validate_input(input_data)` to receive validated and coerced values, converts the inputs into a NumPy array, adjusts dimensions, applies all loaded preprocessor transforms sequentially, and calls the underlying estimator’s `predict()` method.
- `server.py`:
 - `_predict(input_data, endpoint, filename=None)`: Coordinates serving telemetry. It records starting timestamps using `time.perf_counter()`, executes model prediction via `ModelService.predict()`, calculates API response latencies, and logs metrics to the telemetry storage.
 - `/predict` and `/predict/csv` Endpoints: Standard HTTP POST methods. The first endpoint receives a structured JSON payload, while the second parses uploaded CSV files into Pandas DataFrames.
 - `/gui` and `/health` Endpoints: The GUI endpoint returns the dynamic interactive testing form via `_render_gui(schema)`, while the health endpoint serves load balancers with standard status checks.

4.4.3 Deployment Artifact Preparation

Before any service can be deployed, the module must prepare the files that belong to the runtime environment. These include the active `config.json`, model pickle files, service source files, the `accelera` package, runtime requirements, and the generated `Dockerfile`. This preparation step matters because deployment should not depend on scattered files or manual copying. A user should be able to run a command and receive a predictable deployment package.

Algorithm 4.2 describes the artifact preparation process from my implementation perspective. It shows how the module validates model paths, copies the needed runtime files, and generates the files required for container execution.

Algorithm 4.2: Deployment Artifact Preparation

- 1: Input: `config.json`, model artifact paths, deployment source files
 - 2: Output: prepared deployment workspace
-

```

3:
4: 1: Read the active deployment configuration from config.json
5: 2: Extract the configured model artifact paths
6: 3: For each model path do
7: 4:   Resolve the path relative to the deployment workspace
8: 5:   If the artifact does not exist, stop with a clear error
9: 6:   Copy the artifact into the deployment models directory
10: 7:   Rewrite the model reference as a deployment-relative path
11: 8: End for
12: 9: Copy serving files into accelera_deployment/
13: 10: Synchronize the accelera package into the runtime folder
14: 11: Write requirements.txt with serving dependencies
15: 12: Generate a Dockerfile that copies the service, config, and model files
16: 13: Return the prepared package for local or remote deployment

```

This algorithm was important because it turned deployment from a manual process into a repeatable workflow. It also forced me to think carefully about paths. A model path that works during training may not work inside a container unless it is copied and referenced correctly. Handling this properly made the module more reliable and easier for users to follow.

4.4.4 Runtime Schema Validation

To prevent invalid inputs from causing model crashes or silent degradation, incoming payloads are validated against a schema specified in the configuration. The validation layer uses Great Expectations to construct an ephemeral data source context. Columns are validated for expected names, missing entries, unexpected inputs, and value bounds (e.g., minimum and maximum thresholds). Data types are coerced automatically (e.g., matching string, boolean, integer, or floating-point formats) to match the training format, raising a structured `SchemaValidationError` if coercion fails.

This was one of the most important design decisions in the module. A deployed model receives data from outside the training script, so it cannot assume that inputs are correct. The validation layer checks feature names, missing values, extra fields, numeric ranges, allowed values, and data types before inference. Personally, this part taught me that deployment quality is strongly connected to data quality. A model service should not only produce predictions; it should also reject invalid requests clearly.

Algorithm 4.3 summarizes the validation workflow used before prediction.

Algorithm 4.3: Deployment Configuration and Request Validation

```

1: Input: incoming request payload, schema section from config.json
2: Output: validated tabular rows or validation error
3:
4: 1: Convert the payload into a Pandas-compatible table
5: 2: Read the expected feature order from the schema
6: 3: Check for missing columns
7: 4: Check for unexpected columns
8: 5: Coerce each feature according to its configured type
9: 6: Validate required values, numeric ranges, and allowed categories
10: 7: If any validation rule fails, raise SchemaValidationError
11: 8: Return validated rows in the expected model input order

```

The schema validation system is implemented in `schema_validation.py`:

- `InputSchema.__init__(config)`: Extracts features and limits from the configuration. If schema validation is enabled, it requests a Great Expectations context (`gx.get_context()`), generates a unique UUID-based data source, registers a pandas-based `DataFrame` data asset, and configures a whole-dataframe batch definition.

- `InputSchema.validate(input_data)`: Normalizes the input payload into a `DataFrame`. It checks for expected column presence, detects extra or missing features, runs the Great Expectations expectation suite, and executes type coercion.
- `InputSchema.coerce_columns(df, errors)`: Loops through features to coerce types. It casts strings to numerical columns using `pd.to_numeric(..., errors='coerce')`, checks for integer constraints (`value % 1 == 0`), and processes boolean symbols (such as replacing `"true"/"false"` with Python booleans).
- `InputSchema.to_dataframe(input_data)`: Utility method to convert list, dict, or `DataFrame` structures into standard Pandas format.

4.4.5 Telemetry and Request Tracking

Production deployments capture inference requests and runtime metrics using a lightweight tracking layer. When enabled, the tracker logs prediction inputs, model outputs, processing latencies (measured in milliseconds), and timestamps. To avoid performance degradation on the prediction path, logging is written incrementally to a JSON lines (`.jsonl`) file.

Telemetry was included because a deployed system should be observable. During training, the developer can inspect variables directly, but after deployment the service runs through API requests. A tracker gives basic visibility into how many requests were received, how many rows were processed, what statuses occurred, and what the latest event looked like. This made the service more practical and helped connect deployment with monitoring.

Telemetry tracking is implemented in `tracking.py`:

- `PredictionTracker.__init__(config)`: Sets up the file system paths and creates/opens the telemetry log file.
- `PredictionTracker.log_prediction(inputs, outputs, latency, status)`: Formats request parameters, response targets, server latency, and datetime strings into a single-line JSON string and appends it to the logging stream.

4.5 Multi-Target Deployment Orchestration

The orchestration engine automates containerizing and launching the service across three distinct environments. It stages the codebase, copies project dependency requirements, and builds dockerized containers to ensure portability and reproducibility across platforms.

1. **Local Docker**: Generates a clean `Dockerfile` using a Python slim base image, stops any conflicting containers on the target port, builds the container image locally, and runs it.
2. **Heroku**: Authenticates with the Heroku CLI and pushes/releases the Docker container using Heroku's Container Registry.
3. **AWS EC2**: Creates an SSH transport connection and synchronizes the staging directory to the target host using `rsync` (excluding Python cache directories). It installs and starts the Docker daemon, builds the container on the instance, and executes local health check probes before verifying the public web endpoint.

Figure 4.2 shows the deployment-target view of the module. I included this figure because it makes the practical purpose of the orchestration layer clear: the same prepared model service should be able to move from local testing to supported cloud/server environments without requiring the user to manually redesign the serving code for each target.

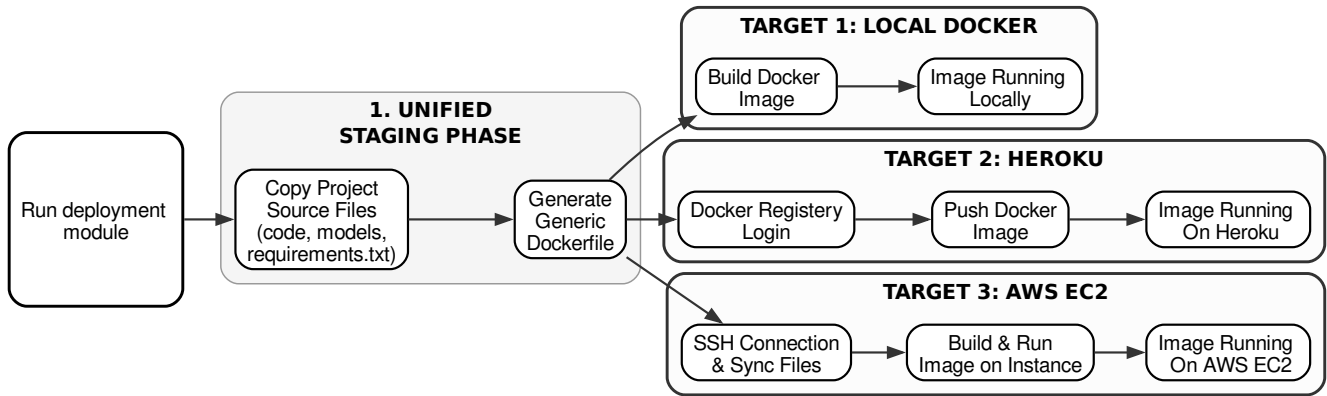


Figure 4.2: Deployment Orchestration Workflow

The orchestration layer was the part where I learned the most about the difference between local development and real deployment. Running a Python file locally is simple compared with preparing a container, exposing a port, sending files to a remote host, building the service on that host, starting a container, and checking that the endpoint responds. The CLI commands were designed to hide much of this complexity from the user while still keeping the workflow understandable.

Algorithm 4.4 describes the remote EC2 connection and synchronization process. It captures the idea that deployment is not just one command; it is a sequence of environment preparation, file transfer, remote execution, and verification.

Algorithm 4.4: Remote Server Connection and Synchronization

- 1: Input: EC2 host, SSH user, optional private key, remote directory
- 2: Output: remote directory containing deployment package
- 3:
- 4: 1: Build the SSH command with host, user, and key options
- 5: 2: Create the remote deployment directory through SSH
- 6: 3: Select Dockerfile, config.json, accelera_deployment/, and models/ as inputs
- 7: 4: Synchronize the selected files using rsync over SSH
- 8: 5: Exclude Python cache files and compiled bytecode
- 9: 6: Return the remote deployment root for container build execution

Algorithm 4.5 shows the service startup and verification workflow. This was important because deployment should not stop after copying files. The module must check that the service actually starts and responds.

Algorithm 4.5: Service Startup and Verification

- 1: Input: remote deployment directory, Docker image name, container name, service port
- 2: Output: running container and verified health endpoint
- 3:
- 4: 1: Enter the remote deployment directory

- 5: 2: Verify that Docker is available or install it when requested
- 6: 3: Build the Docker image from the generated Dockerfile
- 7: 4: Remove any previous container with the same name
- 8: 5: Run the new container with the configured port mapping
- 9: 6: Probe the local /health endpoint inside the remote machine
- 10: 7: Print the public application URL and GUI URL
- 11: 8: Probe the public /health endpoint from the local machine
- 12: 9: If public verification fails, report that the TCP port may need to be opened

These algorithms represent one of the key lessons I gained from the module: deployment must be verifiable. A script that copies files but does not check whether the service is alive leaves the user uncertain. Adding health checks made the workflow more professional and easier to debug.

4.6 Model Version Control (VCS)

The internal VCS provides reproducibility for deployed pipelines. It calculates a stable SHA-1 hash for each commit based on its timestamp and commit message. Models and configurations are archived into an `experiments/` subdirectory. The system maintains a global index (`experiments.json`) containing parent-child commit histories, HEAD references, and active deployed markers, facilitating rollbacks and deployments of historical model versions.

The VCS subsystem was created because source-code version control is not enough for deployment. Git can track Python files, but the deployed state also includes generated model artifacts and the active `config.json`. If these files are changed without a snapshot, it becomes difficult to know which exact model was deployed. The lightweight deployment VCS solves this by committing the model files and configuration together.

Figure 4.3 shows the VCS registry at a higher level. It helped me explain the difference between the active workspace and stored experiment snapshots.

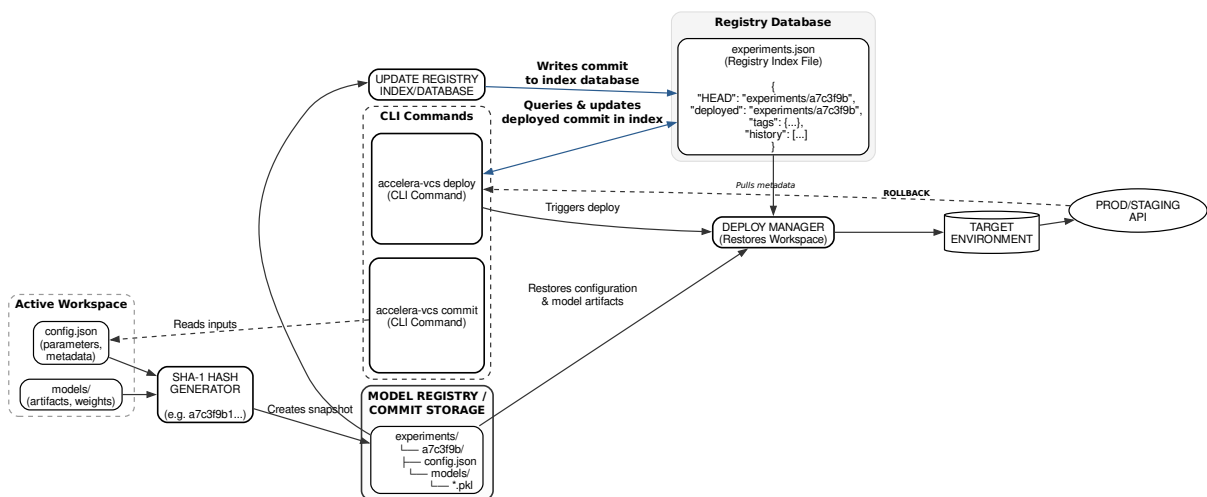


Figure 4.3: Deployment VCS registry structure

The version control subsystem is implemented in `vcs.py`:

- `calculate_hash(timestamp, message)`: Computes the 7-character SHA-1 hash value that acts as the commit identifier.

- **init(args)**: Configures the repository workspace by creating directories and saving the default index (`experiments.json`).
- **commit(args)**: Validates that models exist, generates a commit hash, copies the active configuration and models to `experiments/<hash>/`, and appends the commit entry to the registry.
- **log(args)**: Lists all recorded commits, showing details such as commit hash, creation date, message, and deploy status.
- **show(args)**: Displays the details and settings of a target commit.
- **deploy(args)**: Deploys a specific commit by copying its stored model files and configuration back into the active runtime workspace.

Figure 4.4 focuses on the commit operation. This figure is important because it shows that a deployment commit is more than a message: it captures the active configuration and model files together, then records them in the deployment registry so the state can be understood later.

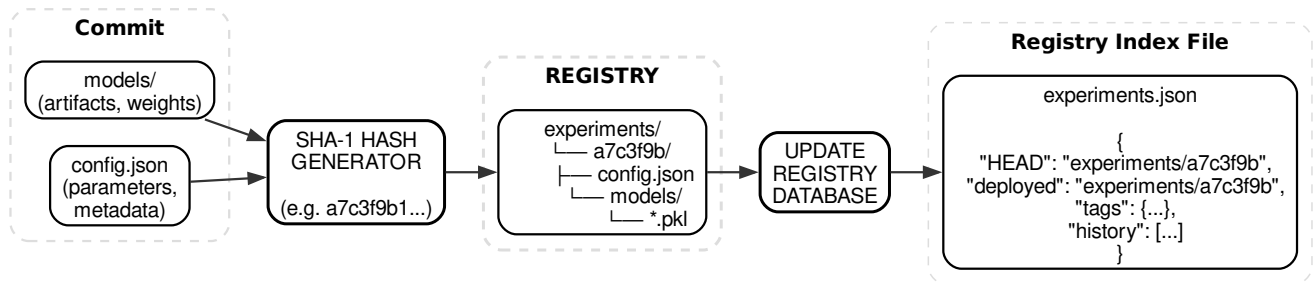


Figure 4.4: VCS Commit Workflow

Figure 4.5 shows the opposite direction of the VCS workflow. Instead of creating a new snapshot, the deployment command restores an existing snapshot into the active workspace. I included this figure because rollback is one of the main practical reasons for adding a deployment-specific VCS.

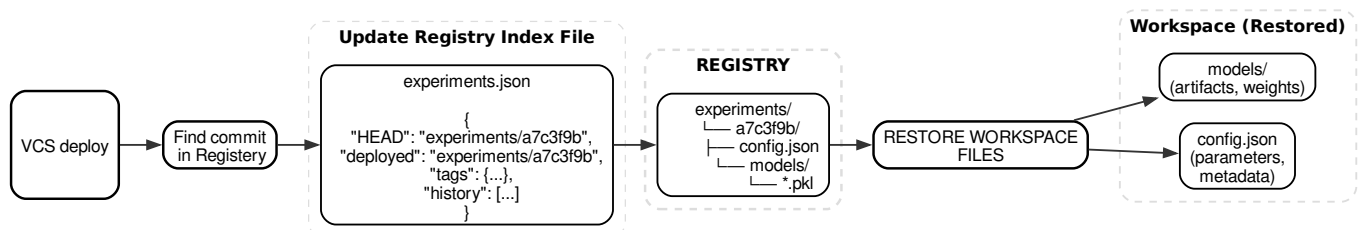


Figure 4.5: VCS Deploy Workflow

From a personal engineering perspective, this subsystem taught me the importance of reproducibility. Deployment is risky when the user cannot return to a previous state. By snapshotting the model files and configuration together, the module gives the user a practical rollback mechanism. This is a small VCS compared with full industrial model registries, but it fits the project scope and solves a real problem inside Accelerera.

4.7 Command-Line Interface and CLI Commands

The orchestrator and version control features are exposed to the user through command-line interfaces inside `deployment.py` and `vcs.py`. The supported commands and their behaviors are:

The CLI was a deliberate design choice because deployment workflows are naturally command-based. Building an image, running a container, logging into Heroku, synchronizing files over SSH, and reading remote logs are all operations that developers usually perform from a terminal. My goal was to organize these operations into clear commands so that the user does not need to remember every low-level step.

Orchestrator Commands

- **prepare**: Prepares the deployment staging area. It copies service modules, synchronizes the `accelera` source library, and generates the serving `Dockerfile` and `requirements.txt`.
- **build**: Builds the local Docker container. It invokes local subprocess shell commands (e.g. `docker build -t <image_name> .`) with an optional `-no-cache` flag.
- **run-local**: Starts the local Docker container. It stops and removes any conflicting containers on the port before launching the container using port mappings.
- **local**: Orchestrates the complete local container pipeline by sequentially running the **prepare**, **build**, and **run-local** steps.
- **heroku-login**: Logs the client environment into the Heroku CLI and Heroku Docker registry.
- **heroku-deploy**: Packages and deploys the container to Heroku. It triggers login, pushes the Docker image to Heroku's registry, and releases the container for traffic.
- **heroku-push** and **heroku-release**: Granular command calls that push the container image and release it separately.
- **ec2-deploy**: Deploys to a remote AWS EC2 instance. It establishes an SSH connection, synchronizes files using `rsync`, compiles/builds the Docker container on the host, runs it, and probes local health targets.
- **ec2-logs** and **ec2-stop**: Operational tools to read Uvicorn outputs and terminate the container on the target EC2 node.

Version Control Commands

- **init**: Sets up a new model version control workspace by initializing directories and registries.
- **commit**: Snapshots active model parameters and configurations under a message flag (**-m**).
- **log**: Prints the history log of all model commits.
- **show**: Prints the configuration dictionary details of a target commit.
- **deploy**: Restores a specified commit back into the staging area for subsequent deployment execution.
- **status**: Prints the current **HEAD** commit and the deployed commit when available.

4.8 Personal Deployment Challenges and Decisions

The Deployment module was the most personally demanding part of this work because it required connecting machine-learning artifacts with backend serving, validation, containers, and remote infrastructure. During implementation, Mazen had to move beyond local model training and understand how a model becomes a long-running service that can receive HTTP requests, validate payloads, return predictions, and remain reproducible across different machines.

One major challenge was organizing the boundary between training artifacts and serving code. The solution was to make **config.json** the central contract between the training side and the deployed service. The service reads the configured model paths, loads all available preprocessors and the final estimator, and applies the same inference path for JSON and CSV requests.

Another challenge was learning how to make deployment repeatable. Instead of relying on manual copying and terminal commands, Mazen implemented the orchestration script to prepare files, write runtime requirements, generate the Dockerfile, build the container, and support local, Heroku, and EC2 workflows. The EC2 path was especially useful as a learning exercise because it required SSH, **rsync**, remote Docker builds, container restart behavior, public ports, and health checks.

The VCS subsystem was an important design decision. Git already tracks source code, but the deployed model state also includes generated model files and runtime configuration. Mazen therefore implemented a lightweight deployment-specific registry that snapshots **config.json** and the active model files, assigns a short hash, records metadata, and supports rollback by restoring a previous snapshot.

Testing the Deployment module also required special care. Many behaviors depend on Docker, Heroku, EC2, SSH, and file-system state, so the verification strategy used temporary workspaces and mocking to test the logic without depending on live cloud services. This made the module more reliable and helped separate the implementation logic from external infrastructure availability.

Chapter 5: System Testing and Verification

5.1 Pipeline Verification

The parallel executor and voting strategies were verified using unit tests.

- **Parallel Execution Tests:** Verify that the topological execution order is preserved, independent branches execute concurrently, and GIL lock release/acquisition behaves correctly.
- **Merge Node Tests:** Validate the correctness of the C++ hard-voting implementation. The test cases verify that the argmax voting correctly matches SciPy baselines for 1D and 3D prediction arrays, and handles the single-model edge case.

5.2 Benchmark Subsystem Verification

The benchmark module tests check that the Node.js API endpoints and database operations behave correctly:

- **Authentication tests:** Check token generation, verify that invalid login details are rejected, and confirm that the signup route hashes passwords correctly.
- **Dashboard routing tests:** Validate user-dashboard calls and verify that admin-level routes reject normal user accounts.
- **Validation tests:** Verify that the server rejects submissions with incorrect row counts or invalid URL structures.

5.3 Deployment Module Verification

The verification of the Deployment module is performed through a comprehensive test suite managed by the `pytest` framework. To ensure high test isolation and allow headless execution in CI/CD pipelines, the test suite leverages extensive mocking via the Python `monkeypatch` utility to decouple tests from system-level configurations, active network services (e.g., Heroku API, AWS EC2 hosts), and runtime dependency installations.

Tests are executed inside temporary filesystem workspaces generated dynamically using the `tmp_path` fixture. This ensures that the generated version control indices, log files, and Docker deployment artifacts do not contaminate the primary workspace.

5.3.1 Subsystem Verification Details

The verification targets the six major components of the deployment module:

1. **Model Version Control (VCS) verification:** Validates the stability of the 7-character short SHA-1 hash calculation, confirming that identical timestamp-message pairs produce identical commit IDs. Tests ensure that `vcs.init()` initializes the

experiments metadata directory structure and saves a clean registry template (`experiments.json`). Commit operations are verified to correctly copy model serialization binaries (e.g., `model.pkl`), snapshot active configurations, update the `HEAD` reference, and append metadata entries. Rollback capabilities are verified by deploying historical commits and confirming that the active runtime files are updated with the corresponding commit snapshots.

2. **Model Service Pipeline verification:** Tests check that the `ModelService` correctly parses configurations, loads active model estimators and preprocessors in the correct sequence, and performs inference. Mock tests confirm that preprocessing steps (such as scaling or encoding) are applied in order before predictions are generated.
3. **Runtime Schema Validation verification:** Validates the runtime behavior of input schema checking. Ephemeral Great Expectations suites are mocked to verify column-presence verification (identifying missing or unexpected fields) and value-bound checking. The data type auto-coercion system is verified by testing how it handles numeric conversions (casting strings to floats or integers) and parsing boolean representations (such as `"true"`, `"false"`, or numeric binary flags).
4. **Web Serving Endpoints verification:** Verifies that the FastAPI application correctly routes incoming HTTP requests. Tests evaluate the `/predict` and `/predict/csv` endpoints for JSON inputs and multi-row CSV file uploads. Endpoint testing checks that success responses return correct JSON structures with HTTP 200 codes, validation errors throw HTTP 422 codes, and server errors return HTTP 500 codes.
5. **Prediction Tracking and Telemetry verification:** Verifies the telemetry logging behavior. Tests ensure that when telemetry is disabled, no disk writes occur. When active, request parameters, model outputs, timestamps, and latency figures are verified to be written as valid JSON lines to the log file. Additionally, tests verify the telemetry reader's ability to aggregate counts (successful runs, error runs, total row counts) and report them dynamically.
6. **Orchestration CLI verification:** Validates port parsing limits (allowing ports between 1 and 65535, while rejecting negative numbers, non-numeric values, or out-of-bound ports). Tests check the preparation task (`prepare`) to verify that it creates a clean staging directory, compiles a standard `Dockerfile` using a Python slim base, and builds a clean `requirements.txt` containing only runtime-specific dependencies.

5.3.2 Deployment Test Results

When executed in an environment with all core dependencies installed, all 51 test cases compile and pass successfully. Table 5.1 summarizes the verification results for the deployment module.

Table 5.1: Deployment Module Test Suite Verification Summary

Test File	Cases	Passed	Failed	Verified Feature
vcs_test.py	14	14	0	Registry snapshots, workspace rollbacks
modelservice_test.py	6	6	0	Sequencing of preprocessor transforms
schema_validation_test.py	8	8	0	Data coercion and column checking
server_test.py	9	9	0	FastAPI routing, telemetry integration
tracking_test.py	3	3	0	Telemetry logging and count aggregation
deployment_test.py	11	11	0	Staging setup, Dockerfile compiling
Total	51	51	0	All deployment checks passed

This successful validation confirms that the deployment engine operates as intended, offering robust models, automated runtime validation, telemetry logging, and repeatable container deployments without regression errors.

Chapter 6: Personal Engineering Reflection

6.1 Individual Challenges

My individual work in Accelerera required moving between different engineering layers. I worked on low-level graph execution behavior, user-facing benchmark interfaces, backend authentication flows, and deployment infrastructure. Each layer introduced a different kind of challenge, and the main difficulty was learning how to connect technical details to the overall value of the project.

6.1.1 Pipeline Module Challenges

The Pipeline Module challenged me to understand how node-based workflows behave internally. At first, it was easier to think of a pipeline as a sequence of steps. However, the graph module required a different mindset. Each node has parents, children, data dependencies, and execution rules. The Merge Node required me to understand how multiple upstream outputs can be combined without losing shape information or producing inconsistent results.

The run-parallel functionality was even more complex because it required thinking about execution order and synchronization. Independent branches can run at the same time, but only when their parent dependencies are complete. This made me realize that parallel execution is not only a performance feature; it is a correctness problem. If the scheduler starts a node too early or releases data too soon, the result may be wrong even if the code compiles. This challenge improved my ability to reason about graph execution and system state.

6.1.2 Benchmark Module Challenges

The Benchmark Module challenged me to think about usability. A backend route may return correct data, but the user still needs a clear interface to understand that data. The user dashboard had to present benchmarks and submissions in a way that felt useful rather than crowded. The admin dashboard had to show system-level information while remaining protected from normal users.

The authentication-related pages also required careful attention. Login and sign-up screens are simple visually, but they connect many parts of the system: user input, backend validation, password hashing, token creation, browser storage, navigation, and permission checks. I learned that a platform feels incomplete if the user flow is confusing, even when the backend logic works correctly.

6.1.3 Deployment Module Challenges

The Deployment module was the largest challenge because it required knowledge beyond ordinary machine-learning training. I had to understand configuration files, serialized model artifacts, serving APIs, Docker containers, SSH connections, remote servers, command execution, public ports, and health checks. At the beginning, these concepts felt

separate. The main challenge was organizing them into one understandable workflow.

Debugging deployment-related behavior also required patience. A problem could come from a missing model path, an incorrect `config.json`, a Dockerfile issue, a port conflict, an unavailable remote host, or a public firewall setting. This taught me that deployment is not only about writing Python code. It is about making a system run correctly in an environment that has many external conditions. Because of this, I focused on clear commands, validation, generated files, and verification steps.

6.2 Learned Technologies and Skills

This project helped me develop both technical and non-technical skills. Technically, I learned how graph-based workflows are represented, how merge nodes combine outputs, how dependency-aware parallel execution works, and why synchronization matters. I also gained practical experience with React dashboards, user/admin interface separation, login and sign-up flows, JWT-based authentication, and backend route design.

The Deployment module taught me the most new technologies. I learned how FastAPI and Uvicorn can expose a model as an API service, how Great Expectations can support runtime input validation, how Docker can package a service into a repeatable container, how SSH and `rsync` are used to communicate with remote EC2 machines, and how command-line scripts can automate deployment workflows. I also learned the importance of configuration management through `config.json` and model artifact paths.

On the non-technical side, I learned how to read and understand an existing codebase, how to debug complex systems with many moving parts, and how to explain technical work in a professional way. Writing the documentation and individual book also helped me connect implementation details to project value. This was important because a graduation project is not only judged by whether the code runs, but also by whether the design decisions, challenges, and outcomes can be communicated clearly.

6.3 Individual Outcomes

The outcome of my work is a stronger and more complete Accelera platform. In the Pipeline Module, the Merge Node made graph workflows more flexible by allowing outputs from different branches to be combined. The shared `runParallel` work with Mohamed Ashraf improved the graph execution capability by allowing independent branches to run concurrently while preserving dependency correctness.

In the Benchmark Module, the user dashboard improved visibility for normal users by showing their benchmarks and submissions. The admin dashboard added a management view for administrator users. The login and sign-up pages helped turn the benchmark module into a more complete platform by supporting account creation, authentication, and role-based access.

The Deployment module was the most important outcome of my individual work. It helped Accelera continue beyond preprocessing, model training, and evaluation. The module packages model artifacts, validates input data, exposes prediction endpoints, records telemetry, stores model snapshots, and supports local and remote deployment workflows. This contribution gives the project a practical final stage: trained models can move toward real usage instead of remaining only local files.

Overall, my work contributed to three levels of completeness: execution completeness

through graph and merge behavior, platform completeness through dashboards and authentication, and lifecycle completeness through deployment. These outcomes made Accelera more useful as an end-to-end machine-learning framework.

Chapter 7: Conclusions and Future Work

7.1 Summary of Contributions

In this graduation project, Mazen Adel designed, implemented, and verified key subsystems for Accelera:

- **Pipeline Execution Layer:** A shared contribution with Mohamed Ashraf to the `runParallel` C++ scheduler utilizing semaphores for hardware thread limitation, safety gates for GPU tasks, and active consumer counting for memory release, paired with Mazen's numpy/scipy-based majority voting nodes.
- **Benchmark Subsystem:** A Node.js Express backend and MongoDB schema model supporting JWT/bcrypt authentication and a React frontend for task management, dashboards, and leaderboard display.
- **Serving and Deployment Infrastructure:** A production-ready FastAPI model server featuring Great Expectations schema validation, async request tracking, model versioning (VCS) hash snapshotting, and automated scripts for Local Docker, Heroku, and remote AWS EC2 deployment targets.

7.2 Future Extensions

Future improvements can focus on the following:

- **Horizontal Scheduling:** Extend the pipeline parallel scheduler to distribute graph branches across different cluster nodes instead of limiting execution to a single host.
- **Robust Encryption:** Improve data security in the benchmark module by encrypting ground truth files on MongoDB and verifying predictions inside sandbox runners.
- **Serverless Serving Targets:** Add serverless deployment capabilities (such as AWS Lambda) to the orchestrator to deploy lightweight models without maintaining active virtual servers.